

C++ std::vector Study Guide: Size, Capacity, and Iterators

1. Introduction to Vector Memory Management

The `std::vector` is a cornerstone of modern C++, providing a dynamic array that manages its own memory lifecycle. For a developer, understanding how it handles internal storage is not merely an academic exercise; it is a non-negotiable requirement for writing performant, memory-safe code. This guide breaks down the mechanics of vector growth and the iterator system used to navigate its contents.

2. Size vs. Capacity: The Fundamentals

Distinguishing between the number of elements and the raw memory allocated is the first step toward mastering vector efficiency.

- **Definitions:**
 - `size()`: The number of actual **constructed elements** currently in the vector.
 - `capacity()`: The total number of elements the vector can hold in its **currently allocated memory** before a reallocation is required.
- **The Bookshelf Mental Model:**
 - **Size**: The number of physical books currently sitting on the shelf.
 - **Capacity**: The total number of slots available on the shelf.
 - `reserve(n)`: Buying a bookshelf with at least n slots, but leaving them empty.
 - `resize(n)`: Actually placing or removing physical books until the shelf contains exactly n.
- **Comparative Table:**

Feature	<code>size()</code>	<code>capacity()</code>
Represents	Count of constructed objects	Total storage allocated
Memory Effect	Constructs/Destructs objects	Allocates/Deallocates raw memory
Focus	Current active contents	Potential for growth
Access Method	<code>v.size()</code>	<code>v.capacity()</code>

Code Example:

```
std::vector<int> v = { 1, 2, 3 };

std::cout << v.size();      // Result: 3 (contains 3 constructed ints)
```

```
std::cout << v.capacity(); // Result: 3 or larger (memory allocated for
future growth)
```

3. Managing Vector Growth: `reserve()` vs. `resize()`

Distinguishing between these two functions is essential for performance optimization.

- **`reserve(n)`**: Increases the **capacity** to at least `n` if it isn't already. It does not create objects or change the size.
- **`resize(n)`**: Directly modifies the **size**. It will construct new elements (using default values) or destroy trailing elements to reach the count of `n`.

CRITICAL WARNING `reserve()` only prepares raw memory; it does **not** create elements. Using an index to access reserved but unconstructed memory (e.g., `v.reserve(10); v[0] = 5;`) is **undefined behavior**. Memory safety hinges on only indexing elements within the range of `0` to `size() - 1`.

Code Comparison:

```
// Case A: Pre-allocating storage
std::vector<int> a;
a.reserve(5);
// a.size() == 0, a.capacity() >= 5. No elements exist yet.

// Case B: Creating elements
std::vector<int> b;
b.resize(5);
// b.size() == 5, b.capacity() >= 5. Elements b[0] through b[4] are
constructed.
```

4. Geometric Growth and Performance

Vector performance is built on the strategy of "geometric growth." To maintain contiguous memory, adding an element when the storage is full requires a "reallocation":

1. Allocate a larger block of memory.
2. Move or copy existing elements to the new block.
3. Free the old memory block.

Growth Patterns: Reallocation is expensive. To minimize this cost, vectors grow in "chunks." The exact factor is implementation-dependent, but common sequences include:

- **Factor of 2:** `0 -> 1 -> 2 -> 4 -> 8 -> 16 -> 32`
- **Factor of ~1.5:** `0 -> 1 -> 2 -> 3 -> 4 -> 6 -> 9 -> 13`

Optimization: If you know you will be adding 1,000,000 elements via `push_back()` or `emplace_back()`, use `reserve(1000000)` first. This ensures only one allocation occurs, making the operation amortized $O(1)$ rather than $O(n)$.

Shrinking: Capacity does not automatically shrink. While `shrink_to_fit()` exists to request a reduction in capacity to match size, it is a non-binding request to the compiler.

5. Iterators: The Navigation System

An iterator is an object that acts as a "bookmark" or "pointer-like tool," allowing you to traverse a container one element at a time.

- **The Finger-and-Boxes Mental Model:** Imagine the vector as a row of boxes. An iterator is your finger pointing at one specific box. [10] [20] [30] [40] [STOP]
- **Key Positions:**
 - `begin()`: Places your "finger" on the first box (10).
 - `end()`: Places your "finger" on the **stop marker** just *after* the last box.
- **Operations:**
 - `*it`: Read the value inside the box your finger is pointing to.
 - `++it`: Move your finger one box to the right.

Example:

```
std::vector<int> v = { 10, 20, 30 };
auto it = v.begin();
std::cout << *it; // Output: 10
```

6. Looping with Iterators

Iterators allow for a uniform way to process all elements in a container.

- **The Classic Iterator Loop:**
- **Modern Alternative:** Range-based `for` loops (`for (int x : v)`) are cleaner and use iterators "behind the scenes."
- **Code Trace (Modifying Values):**
- **Trace for {1, 2, 3}:**
 1. **Iteration 1:** `it` points to 1. `*it` (the value 1) is multiplied by 10. Vector: {10, 2, 3}.
 2. **Iteration 2:** `it` moves to 2. `*it` becomes 20. Vector: {10, 20, 30}.
 3. **Iteration 3:** `it` moves to 3. `*it` becomes 30. Vector: {10, 20, 30}.
 4. **Exit:** `it` reaches `v.end()` (the stop marker), and the loop terminates.

7. Specialized Iterator Types and Operations

- **Const Iterators:** Use `cbegin()` and `cend()` for read-only access. Attempting `*it = 5` will cause a compiler error, protecting your data.
- **Reverse Iterators:** `rbegin()` points to the last element, and `rend()` points to the position before the first. `++it` moves backward through the vector.
- **Vector-Specific Arithmetic:** Because vector memory is contiguous, iterators support "strong" operations:
 - `it + 2`: Jump forward two boxes.
 - `--it`: Move to the previous box.
 - `it1 == it2`: Check if two iterators are pointing at the same box.
 - `v.end() - v.begin()`: Returns the total number of elements.

8. The Danger of Iterator Invalidation

One of the most critical rules in C++ is that modifying a vector can make your "fingers" (iterators) point to garbage memory.

- **The Rule:** Operations that cause reallocation (`push_back`, `emplace_back`, `insert`, `resize`) can move the entire array to a new memory location. When this happens, any existing iterators, pointers, or references are rendered **invalid**.
- **Code Example of Failure:**
- **Safety Advice:**
 - Always assume iterators are unsafe after a vector changes size.
 - **Check Return Values:** Functions like `insert` and `erase` return a **new, valid iterator** to the updated position. Use these return values to re-sync your iterators.

9. Iterators in the STL Ecosystem

Iterators are the "standard language" that connects containers to algorithms.

Practical Example:

```
std::sort(v.begin(), v.end());
```

In this call, the `std::sort` algorithm doesn't need to know it's working with a `vector`. It simply uses the range provided by the iterators (from the first box up to, but not including, the stop marker) to perform its work.

10. Study Guide Summary: Top 7 Takeaways

1. **Size vs. Capacity:** `size()` is the current book count; `capacity()` is the total shelf space available.
2. **reserve vs. resize:** `reserve()` allocates raw memory (capacity); `resize()` constructs actual objects (size).
3. **Growth Strategy:** Vectors grow geometrically (e.g., doubling) to ensure `push_back` is efficient (amortized $O(1)$).
4. **begin and end:** `begin()` is the first element; `end()` is a non-dereferenceable "stop marker" past the last element.
5. **Iterator Movement:** Use `++it` to move forward, `--it` to move backward, and `*it` to access the value.
6. **Invalidation Rule:** If the vector reallocates (due to size changes), all existing iterators and pointers are potentially broken.
7. **Algorithm Integration:** Iterators are the universal interface for C++ Standard Library algorithms like `std::sort` and `std::find`.